



An-Najah National University

Faculty of Engineering & Information Technology

Dos-Project - Part 2

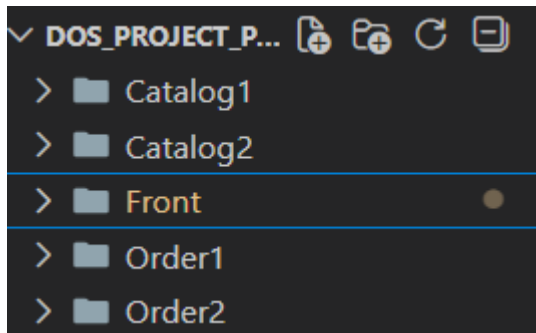
Jawad Hamdan – 12113704

Mohammad Khalaf – 12112969

1. Introduction

This report presents the key features implemented in Phase Two of the project, including an in-memory caching system, a load-balancing strategy, catalog and order services replication using a microservices-based architecture and RESTful APIs.

2- Project Structure : we have replicated the Catalog service and its DB file and the same were done for the Order service as the following photo :



3-Cache:

for read operations. A Map-based cache was added to store responses for catalog requests. Cached entries are keyed by the query (e.g., `search:<topic>` and `info:<id>`) and include an expiration timestamp (TTL). For each incoming search request, the frontend first calls `cacheGet()` to check whether a valid (non-expired) entry exists. If a cache hit occurs, the cached response is returned immediately. Otherwise, the frontend forwards the request to a catalog replica (selected using round-robin load balancing), stores the returned data using `cacheSet()`, and then returns it to the client. The cache is bounded by a maximum size and evicts the oldest entry when full. Additionally, a server-push invalidation endpoint (`/cache/invalidate/:id`) is provided so the catalog service can invalidate cached item information before write operations to maintain consistency.

```

const cache = new Map(); // key -> { value, expiresAt }

// search => uses the cache and it could hit or mis
app.get("/search/:topic", async (req, res) => {
  const topic = String(req.params.topic || "");
  const key = `search:${topic.toLowerCase()}`;

  const hit = cacheGet(key);
  if (hit) return res.json({ cached: true, data: hit });

  const catalogURL = pickCatalog();
  const r = await httpJSON(`${catalogURL}/search/${encodeURIComponent(topic)}`);
  if (!r.ok) return res.status(r.status).json(r.data ?? { error: "Catalog error" });

  cacheSet(key, { (property) cached: boolean });
  res.json({ cached: false, replica: catalogURL, data: r.data });
});

```

4. load balancing : requests across multiple replicas of the services on the back. The Round Robin algorithm was selected due to its simplicity and low overhead. For each incoming search request, the frontend selects one replica in a cyclic manner, ensuring that consecutive requests are forwarded to different replicas. This approach balances the load among the replicas and prevents any single replica from becoming a bottleneck.

```

const CATALOG_REPLICAS = (process.env.CATALOG_REPLICAS || "http://localhost:4001,http://localhost:4003")
  .split(",")
  .map(s => s.trim())
  .filter(Boolean);

const ORDER_REPLICAS = (process.env.ORDER_REPLICAS || "http://localhost:4002,http://localhost:4004")
  .split(",")
  .map(s => s.trim())
  .filter(Boolean);

```

The code that actually implements it :

```
let catIdx = 0;
let ordIdx = 0;

function pickCatalog() {
  const url = CATALOG_REPLICAS[catIdx % CATALOG_REPLICAS.length];
  catIdx = (catIdx + 1) % CATALOG_REPLICAS.length;
  return url;
}

function pickOrder() {
  const url = ORDER_REPLICAS[ordIdx % ORDER_REPLICAS.length];
  ordIdx = (ordIdx + 1) % ORDER_REPLICAS.length;
  return url;
}
```

Note: To ensure consistency between the primary replica and its backup, All write operations are handled exclusively by the primary replica, which then propagates updates to the backup replica to ensure they are at the same state.

```
async function replicateToPeer(op) {
  if (!IS_PRIMARY) return;
  try {
    await fetch(`${PEER_URL}/internal/replicate`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(op),
    });
  } catch (e) {
    console.warn(`[Catalog:${PORT}] replicateToPeer failed:`, e?.message || e);
  }
}
```

```
app.put("/update/price", async (req, res) => {
  const { id, price } = req.body || {};
  if (!Number.isInteger(id) || !Number.isFinite(price) || price < 0)
    return res.status(400).json({ error: "Invalid id or price" });

  await invalidateCache(id);
  const local = applyWrite({ type: "update_price", id, price });
  if (!local.ok) return res.status(local.code).json({ error: local.error });

  await replicateToPeer({ type: "update_price", id, price });
  res.json({ ok: true, primary: IS_PRIMARY, port: PORT });
});
```

5-Cache Invalidation:

To prevent stale data from being served by the frontend cache, a server-push cache invalidation mechanism was implemented. Before executing any write operation, the back services notifies the frontend to invalidate the affected cache entries.

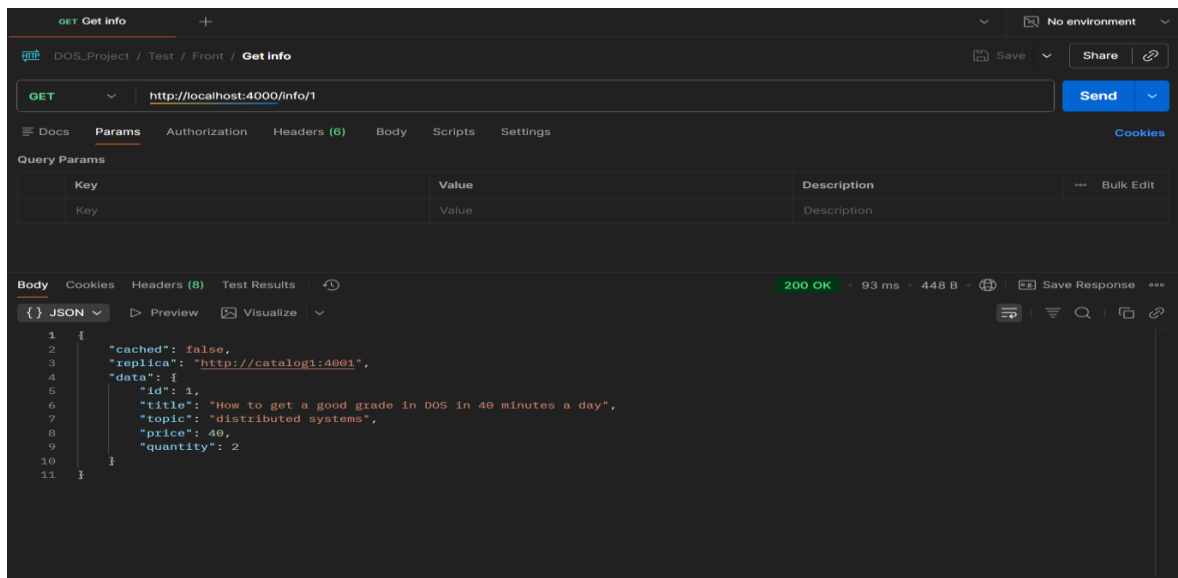
=> Before any write it invalidate the cache at frontend service

```
// Invalidate endpoint => server-push implementation
app.post("/cache/invalidate/:id", (req, res) => {
  const id = Number(req.params.id);
  if (!Number.isInteger(id) || id <= 0) return res.status(400).json({ error: "Invalid id" });
  invalidateBook(id);
  res.json({ ok: true, invalidated: id });
});
```

Note : the Order service communicate with the primary Catalog service and any update the primary Catalog updates its backup.

6-Experiments :

Not Cached (miss) => 93m



GET Get info

DOS_Project / Test / Front / Get Info

GET http://localhost:4000/info/1

Send

Query Params

Key	Value	Description
Key	Value	Description

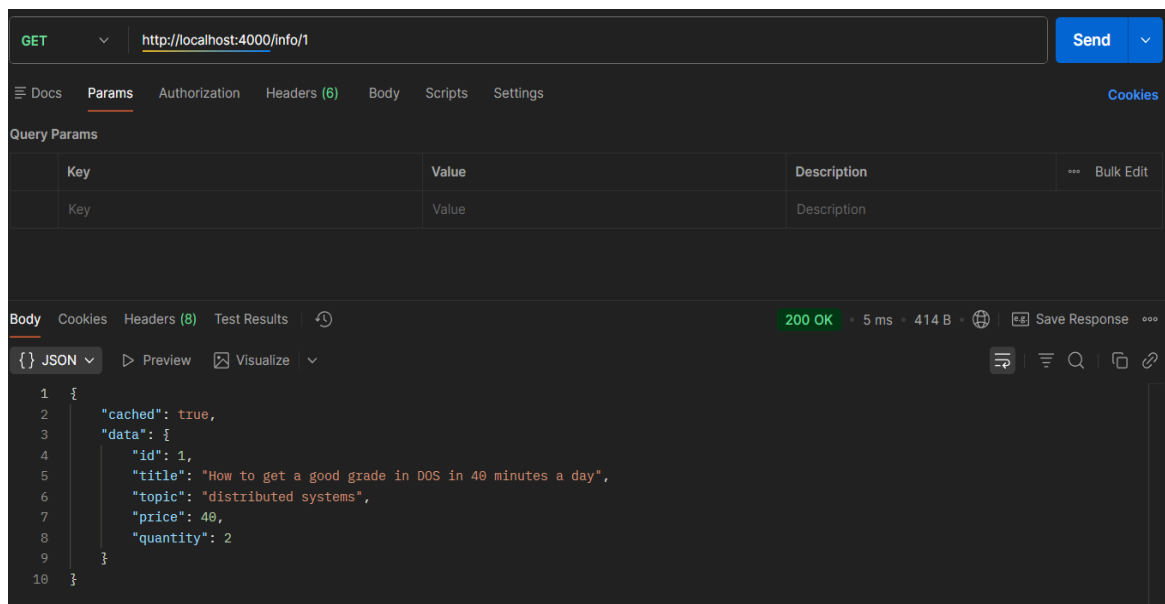
Body Cookies Headers (8) Test Results

200 OK - 93 ms - 448 B

Save Response

```
{
  "cached": false,
  "replica": "http://catalog1:4001",
  "data": {
    "id": 1,
    "title": "How to get a good grade in DOS in 40 minutes a day",
    "topic": "distributed systems",
    "price": 40,
    "quantity": 2
  }
}
```

Cached (hit) => 5ms



GET http://localhost:4000/info/1

Send

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (8) Test Results

200 OK - 5 ms - 414 B

Save Response

```
{
  "cached": true,
  "data": {
    "id": 1,
    "title": "How to get a good grade in DOS in 40 minutes a day",
    "topic": "distributed systems",
    "price": 40,
    "quantity": 2
  }
}
```

Not cached (miss) => 23ms

GET

http://localhost:4000/search/How to get a good grade in DOS in 40 minutes a day

Send

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

Cookies

Headers (8)

Test Results

200 OK

23 ms

326 B

Save Response

JSON

Preview

Visualize

```
1 {
2   "cached": false,
3   "replica": "http://catalog1:4001",
4   "data": []
5 }
```

Cached (hit) => 4ms

GET

http://localhost:4000/search/How to get a good grade in DOS in 40 minutes a day

Send

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

Cookies

Headers (8)

Test Results

200 OK

4 ms

292 B

Save Response

JSON

Preview

Visualize

```
1 {
2   "cached": true,
3   "data": []
4 }
```

Conclusion : implementing caching significantly reduced the average response time

Note : The first time the topic is searched, the data is retrieved from the catalog service (cache miss). On the second search for the same topic, the data is fetched directly from the cache, demonstrating improved efficiency.

To evaluate the effectiveness of the in-memory cache, response times were measured for repeated read requests. The first request resulted in a cache miss and required communication with the catalog service, leading to higher latency. Subsequent requests were served directly from the cache, significantly reducing response time.

Ports and Services :

Front : 4000

Catalog 1 : 4001

Catalog 2 : 4003

Order 1 : 4002

Order 2: 4004

Optional Part :

Containers [Give feedback](#)

Container CPU usage

0.00% / 1600% (16 CPUs available)

Container memory usage

75MB / 7.43GB

Show charts

Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	dos_project_part1	-	-	-	0%	35 minutes ago	
☐	catalog2-1	a96512199757	dos_project_part1-catalo	4003:4003	0%	35 minutes ago	
☐	catalog1-1	3646c6067780	dos_project_part1-catalo	4001:4001	0%	35 minutes ago	
☐	order1-1	974efadf68b2	dos_project_part1-order	4002:4002	0%	35 minutes ago	
☐	order2-1	19681c5ec136	dos_project_part1-order	4004:4004	0%	35 minutes ago	
☐	front-1	aa6051998a18	dos_project_part1-front	4000:4000	0%	35 minutes ago	

